```
<body ng-init=" people=[{name:'Tony',branch:'CSE'},
          {name:'Santhosh', branch:'EEE'},
          {name:'Manisha', branch:'ECE'}];">
Name: <input type="text" ng-model="name"/>
<li ng-repeat="person in people | filter: name"> {{person.
name }} - {{person.branch}}
</li>
</body>
```

Advanced features

Controllers: To bring some more action to our app, we need controllers. These are JavaScript functions that add behaviour to our app. Let's make use of the ngController directive to bind the controller to the DOM:

```
<body ng-controller="ContactController">
     <input type="text" ng-model="name"/>
     <button ng-click="disp()">Alert !</button>
     <script type="text/javascript">
       function ContactController($scope) {
           $scope.disp = function( ){
           alert("Hey " + $scope.name);
                  };
                        }
             </script>
</body>
```

One term to be explained here is *'$scope'*. To quote from the developer guide: "Scope is an object that refers to the application model." With the help of scope, the model variables can be initialised and accessed. In the above example, when the button is clicked the *disp( )* comes into play, i.e., the scope is assigned with a behaviour. Inside *disp( )*, the model variable name is accessed using scope.

Views and routes: In any usual application, we navigate to different pages. In an SPA, instead of pages, we have views. So, you can use views to load different parts of your application. Switching to different views is done through routing. For routing, we make use of the *ngRoute* and *ngView* directives:

```
var miniApp = angular.module( 'miniApp', ['ngRoute'] );

miniApp.config(function( $routeProvider ){
     $routeProvider.when( '/home', { templateUrl:
'partials/home.html' } );
     $routeProvider.when( '/animal', {templateUrl:
'partials/animals.html' } );
     $routeProvider.otherwise( { redirectTo: '/home' } );
});
```

ngRoute enables routing in applications and *$routeProvider* is used to configure the routes. *home.html* and *animals.html* are examples of 'partials'; these are files that will be loaded to your view, depending on the URL passed. For example, you could have an app that has icons and whenever the icon is clicked, a link is passed. Depending on the link, the corresponding partial is loaded to the view. This is how you pass links:

```
<a href='#/home'><img src='partials/home.jpg' /></a>
<a href='#/animal'><img src='partials/animals.jpg' /></a>
```

Don't forget to add the 'ng-view' attribute to the HTML component of your choice. That component will act as a placeholder for your views.

```
<div ng-view=""></div>
```

Services: According to the official documentation of AngularJS, "Angular services are substitutable objects that are wired together using dependency injection (DI). You can use services to organise and share code across your app." With DI, every component will receive a reference to the service. Angular provides useful services like *$http, $window,* and *$location*. In order to use these services in controllers, you can add them as dependencies. As in:

```
var testapp = angular.module( 'testapp', [ ] );
   testapp.controller ( 'testcont', function( $window ) {
            //body of controller
});
```

To define a custom service, write the following:

```
testapp.factory ('serviceName', function( ) {
var obj;
return obj;       // returned object will be injected to
the component
      //that has called the service
});
```

Testing

Testing is done to correct your code on-the-go and avoid ending up with a pile of errors on completing your app's development. Testing can get complicated when your app grows in size and APIs start to get tangled up, but Angular has got its own defined testing schemes. Usually, two kinds of testing are employed, unit and end-to-end testing (E2E). Unit testing is used to test individual API components, while in E2E testing, the working of a set of components is tested.

The usual components of unit testing are *describe( ), beforeEach( )* and *it( )*. You have to load the angular module before testing and *beforeEach( )* does this. Also, this function